Assignment 1:

1. product.py:

```python
import re

class Product:
    _total_count = 0

    def __init__(self, name, sku, price, quantity):
        if not self.is_valid_sku(sku):
            raise ValueError(f"Invalid SKU: {sku}")
        self.name = name
        self.sku = sku
        self.price = price
        self.quantity = quantity
        Product._total_count += 1

    def __repr__(self):
        return f"Product({self.name!r}, {self.sku!r}, {self.price}, {self.quantity})"

    @property
    def price(self):
        return self._price

    @price.setter
    def price(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("Price must be a number")
        if value < 0:
            raise ValueError("Price cannot be negative")
        self._price = float(value)

    @property
    def quantity(self):
        return self. quantity
```

```
@quantity.setter
def quantity(self, value):
    if not isinstance(value, int) or isinstance(value, bool):
        raise TypeError("Quantity must be an integer")
    if value < 0:
        raise ValueError("Quantity cannot be negative")
    self._quantity = value

@property
def line_value(self):
    return self.price * self.quantity

@property
def is_low_stock(self):
    return self.quantity < 5

@classmethod
def from_csv_line(cls, line):
    parts = [p.strip() for p in line.split(",")]
    return cls(parts[0], parts[1], float(parts[2]), int(parts[3]))

@staticmethod
def is_valid_sku(sku):
    return bool(re.fullmatch(r"[A-Za-z]{3}\d{3}", sku))

@classmethod
def count(cls):
    return cls._total_count
```

2. inventory.py

Python

```python
from product import Product

class Inventory:
    def __init__(self):
        self.products = {}

    def add(self, product):
        self.products[product.sku] = product

    def remove(self, sku):
        return self.products.pop(sku, None)

    def find(self, sku):
        return self.products.get(sku)

    def total_value(self):
        return sum(p.line_value for p in self.products.values())

    @classmethod
    def from_lines(cls, lines):
        inv = cls()
        for line in lines:
            inv.add(Product.from_csv_line(line))
        return inv
```

3. main.py

```python
from inventory import Inventory

SAMPLE = [
    "Milk, MLK001,1.25,12",
    "Bread, BRD002,0.90,3",
    "Eggs, EGG003,2.40,30",
    "Coffee, COF004,7.50,2",
]

def main():
    inv = Inventory.from_lines(SAMPLE)
    print(f"{'Product':<10} | {'SKU':<8} | {'Price':<6} | {'Qty':<4} | {'Line Value'
    print("-" * 50)
    for p in inv.products.values():
        print(f"{p.name:<10} | {p.sku:<8} | ${p.price:<5.2f} | {p.quantity:<4} | ${p
    print("-" * 50)
    print(f"Total Inventory Value: ${inv.total_value():.2f}")
    low_stock = [p.name for p in inv.products.values() if p.is_low_stock]
    print(f"Low Stock Items (<5): {', '.join(low_stock)}")

if __name__ == "__main__":
    main()
```

4. README.md

```markdown
# MiniMart Inventory System

## Overview
A simple Python 3.10 object-oriented inventory management system for MiniMart.

## Features
- OOP structure separating `Product` and `Inventory`.
- Encapsulation with property setters for strict type and value validation.
- Factory class methods and SKU pattern static validation.

## How to Run
Run the main demonstration script:
```bash
python main.py
```

```bash
pytest
```
```

test_product.py`

```
### 5. `test_product.py` (5+ اختياري للبونص)

```python
import pytest
from product import Product

def test_valid_product():
    p = Product("Milk", "MLK001", 1.25, 12)
    assert p.line_value == 15.0
    assert not p.is_low_stock

def test_invalid_price():
    with pytest.raises(ValueError):
        Product("Milk", "MLK001", -1.0, 10)
    with pytest.raises(TypeError):
        Product("Milk", "MLK001", "1.25", 10)

def test_invalid_quantity():
    with pytest.raises(ValueError):
        Product("Milk", "MLK001", 1.25, -5)

def test_invalid_sku():
    with pytest.raises(ValueError):
        Product("Milk", "INVALID", 1.25, 10)
```

```
def test_low_stock():
    p = Product("Bread", "BRD002", 0.90, 3)
    assert p.is_low_stock is True
```
```